

Two-Agent IT Support System: A2A, RAG and Playwright

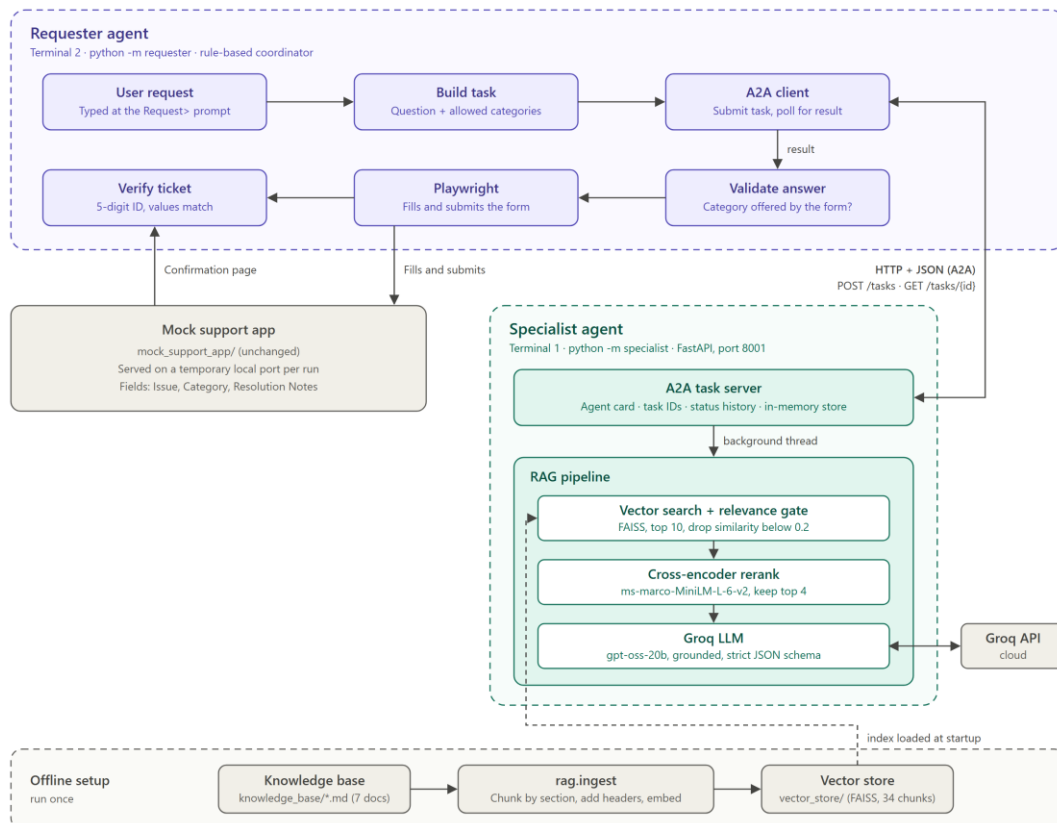
1. System overview

The system turns a plain-English IT support request into a verified support ticket, using two cooperating agents:

- **Requester Agent:** a rule-based coordinator. It receives the user's request, delegates the analysis to the Specialist over an A2A-style HTTP protocol, and waits for the result. It then uses Playwright to file a ticket in the course's mock support app and checks that the ticket was recorded correctly.
- **Specialist Agent:** a web service that answers with Retrieval-Augmented Generation over the course knowledge base. For each request it returns a ticket category, resolution notes, an escalation flag, and the knowledge base used.

The Specialist's answer directly drives the browser. The form's **Category** dropdown and **Resolution Notes** field are filled from it, and the notes cite the knowledge base.

2. System architecture



(Generated visual image of the architecture using Claude)

Layer	Package
Protocol	A2A
Specialist Agent	specialist/
RAG	rag/
Requester Agent	requester/
Browser Automation	automation/

Technology stack:

- Python 3.13, with FastAPI and uvicorn for the Specialist server, and requests for the client.
- LangChain, FAISS, sentence-transformers and Groq for RAG.
- Playwright (Chromium) for browser automation.
- Embedding and reranking run locally on the CPU. Only generation calls Groq API

3. End-to-end workflow

Example request: *"My keyboard has stopped working."*

#	Step	Component
1	The user types the request at the Requester's <code>Request></code> prompt	Requester
2	The Requester reads the Specialist's agent card at <code>./well-known/agent.json</code> , which confirms the Specialist is reachable	Requester → Specialist
3	The Requester builds a task with the question plus the form's four allowed categories, and sends <code>POST /tasks</code>	Requester
4	The Specialist stores the task and immediately replies <code>202 {task_id, status: "submitted"}</code> . The RAG work starts on a background thread and the status becomes <code>working</code>	Specialist
5	Vector search, relevance gate, cross-encoder rerank and Groq LLM produce a grounded JSON answer	Specialist / RAG
6	The Specialist maps the answer's category onto the form's options, saves the result, and marks the task <code>completed</code> , or <code>failed</code> with an error code	Specialist
7	Meanwhile the Requester polls <code>GET /tasks/{id}</code> every 1 s, until the task is <code>completed</code> or <code>failed</code> or its timeout (30 s by default) runs out	Requester
8	The Requester checks the category exists on the form and the resolution isn't empty	Requester

9	Playwright opens the form, fills Issue Description (the user's text), Category and Resolution Notes (both from the Specialist), and submits	Playwright
10	Playwright checks the confirmation: ticket ID is 5 digits, and the category and resolution shown match what was entered. The Requester prints <code>SUCCESS: Ticket #27201 submitted and verified (Hardware).</code>	Playwright

4. A2A protocol

The protocol is plain HTTP and JSON, modelled on the A2A task lifecycle (task ID, status transitions, artifact retrieval).

Endpoints

Endpoint	Purpose	Response
GET /.well-known/agent.json	Agent card: name, description, skill (<code>triage_ticket</code>), endpoints, statuses	200
POST /tasks	Submit a task	202 { <code>task_id</code> , <code>status</code> : "submitted", <code>created_at</code> }, returned immediately
GET /tasks/{ <code>task_id</code> }	Status check and result retrieval	200 with status, timestamped history, result or error; 404 for an unknown ID

Messages

Task request (Requester → Specialist):

```
{
  "question": "My keyboard has stopped working.",
  "allowed_categories": ["Account Access", "Hardware", "Software", "Network"],
  "metadata": {}
}
```

Completed task (Specialist → Requester)

```
{
  "task_id": "435163241de0",
  "status": "completed",
  "history": [{"status": "submitted", "timestamp": "..."}, {"status": "working", "...": "..."},
{"status": "completed", "...": "..."}],
  "result": {
    "category": "Hardware",
    "kb_category": "Hardware",
    "resolution": "The technician should first confirm that the keyboard is properly connected
...",
    "escalate": false,
    "confidence": 0.465,
    "sources": [{"source": "hardware.md", "section": "Symptoms", "chunk_id":
"hardware:symptoms:0",
      "score": 0.465, "rerank_score": -3.552}]
  }
}
```

Failed task:

"status": "failed" with, for example, "error": {"code": "insufficient_context", "message": "..."}.

Implementation

- **Server** (a2a_protocol/server.py):
 - In memory task store.
 - Each submitted task runs on its own background thread, so POST /tasks never waits for the RAG work.
 - The worker wraps the agent's handler. A deliberate TaskFailure(code, message) becomes failed with that code. Any unexpected exception also becomes failed (internal_error), so a task can never be left stuck in working.
- **Client** (a2a_protocol/client.py):
 - Every HTTP call has a 5s timeout.
 - wait_for_result() polls until the task finishes or the overall deadline passes.
 - Errors are typed:
 - AgentUnavailableError
 - TaskNotFoundError
 - TaskFailedError
 - TaskTimeoutError

5. RAG approach

Stage	Implementation
Load	7 markdown documents from <code>knowledge_base/</code>
Chunk	Split on # and ## headers, one chunk per section, giving 34 chunks . A character splitter (800 characters, 100 overlap) is only a fallback for unusually long sections.
Enrich	Contextual Chunk Headers (below)
Embed	<code>sentence-transformers/all-MiniLM-L6-v2</code>
Store	FAISS with inner product, so scores are cosine similarity
Retrieve	Top 10 candidates. Any candidate below a similarity of 0.2 is discarded.
Rerank	Cross-encoder <code>ms-marco-MiniLM-L-6-v2</code> ; keep the top 4
Generate	Groq <code>openai/gpt-oss-20b</code> , temperature 0, strict JSON schema, retrieved passages only
Return	Category, resolution, escalation flag, confidence, cited sources

Advanced technique 1: Contextual Chunk Headers (rag_techniques #10)

Every knowledge-base document shares the same layout: Description, Symptoms, Troubleshooting Steps, Resolution, Escalation. A chunk containing only a Troubleshooting section ("restart the computer...") looks much the same in the Hardware, Network and Software docs. Before embedding, each chunk gets a header:

Document: Network Connectivity

Section: Symptoms

Ticket Category: Network

Measured effect (python -m rag.ingest --compare, on the course's test_cases.json):

- With headers, 5/5 requests retrieved the correct category first.
- Without headers, 4/5 did. *"My laptop won't connect to Wi-Fi"* retrieved hardware:troubleshooting_steps first without headers, and network:symptoms with them.

Advanced technique 2: Intelligent Reranking (rag_techniques #17)

The embedding model encodes the question and each chunk separately, so a chunk that merely sounds similar can outrank the right one. A cross-encoder reads the question and chunk together and scores their relevance directly.

Grounding

- The LLM sees only the 4 retrieved passages. The prompt forbids adding steps, contacts or tools the passages don't mention.
- The JSON schema is **built for each request**. kb_category is an enum of the categories present in the retrieved passages, and cited_chunks is an enum of their chunk IDs. Groq enforces the schema while generating, so the model cannot invent a category or cite a passage it wasn't given.
- Every answer is validated again in code. Invalid output gets one retry, then fails with llm_invalid_output.

Results

Full pipeline on the 5 course test cases plus 6 edge cases:

Request	Category	Escalate	Correct?
Forgot password, cannot log in	Account Access	No	Yes
Laptop won't connect to Wi-Fi	Network	No	Yes
Keyboard stopped working	Hardware	No	Yes
Cannot install software	Software	No	Yes
Company email not synchronizing	Email	No	Yes

Someone hacked my account	Security	Yes	Yes
Suspicious email with a weird link	Security	Yes	Yes
Monitor flickering (weakest match, 0.28)	Hardware	No	Yes
Whole office lost internet	Network	Yes	Yes
Capital of France?	Rejected: <code>insufficient_context</code>	n/a	Yes
Office printer color?	Rejected: <code>insufficient_context</code>	n/a	Yes

The result was 11/11, at about 1s per answer. The escalate flag was true only where the knowledge base's Escalation criteria apply: suspected compromise, phishing, and multiple users affected.

6. Playwright implementation

Implemented in `automation/ticket_form.py` and `automation/app_server.py`.

1. **Serve the app.** The mock app is served from `mock_support_app/` by Python's built-in HTTP server, on a free local port, for the duration of each run. The app's files are unchanged.
2. **Open and confirm.** Playwright navigates to the form and waits for the "Submit a Support Ticket" heading.
3. **Check the category first.** The dropdown's options are read from the live page. If the requested category isn't among them, the run stops before anything is typed, with an `UnsupportedCategoryError`.
4. **Fill the fields.** Elements are located by **accessible label and role**, e.g. `get_by_label("Resolution Notes")` and `get_by_role("button", name="Submit Ticket")`, rather than CSS IDs. This matches how a user or screen reader finds the fields.
5. **Submit.** Playwright clicks the button, then checks for the form's own error message (`role="alert"`).
6. **Verify.** It waits for the confirmation panel (`role="status"`) and checks:
 - the "Ticket Submitted Successfully!" heading appears;
 - the ticket ID matches
 - the displayed category equals the one selected

- the displayed resolution equals the text entered while ignoring whitespace

Any mismatch raises `TicketSubmissionError`, naming the problem.

7. Failure handling

Situation	Detected by	Outcome
Specialist not running	A2A client	Specialist Agent is unavailable
Specialist too slow	Requester's polling deadline	TIMEOUT ... (last status: working), no ticket
Question unrelated to the knowledge base	Similarity gate (< 0.2)	failed: insufficient_context, no LLM call made
Passages retrieved but not relevant	LLM returns <code>answerable: false</code>	failed: insufficient_context
API key missing or invalid	Generator	failed: missing_api_key / llm_auth_error
Rate limit, network error, Groq	Generator (after one retry)	failed: llm_unavailable
Malformed LLM output	Generator (after one retry)	failed: llm_invalid_output
Unexpected Specialist error	A2A server	failed: internal_error
Unknown task ID	A2A server	HTTP 404
Category not offered by the form (Email, Security)	Requester validation	CANNOT FILE TICKET ... Route this ticket manually
Form error or confirmation mismatch	Playwright	Ticket submission did not verify
Browser cannot launch	Playwright	Clear message with the install command
Any unforeseen error	Interactive session	UNEXPECTED ERROR ...; the session continues

For the demo, `/slow` makes the Specialist sleep 60 s so the Requester times out, and `/fail` makes it fail immediately. The single-request equivalents are `--simulate slow` and `--simulate fail`.

8. Design decisions

1. **Plain HTTP and JSON rather than an A2A SDK.** It's transparent and easy to inspect: FastAPI's /docs page lets you submit and poll tasks by hand. It still implements the core A2A ideas: an agent card, task IDs, asynchronous acknowledgment, a status lifecycle and result retrieval.
2. **Protocol kept separate from agent logic.** `a2a_protocol/` is generic, and the Specialist plugs in one handler function. When RAG replaced the early retrieval-only placeholder, the protocol code didn't change.
3. **One endpoint for status and result.** `GET /tasks/{id}` returns both, as A2A's `tasks/get` does. The Requester needs a single call per poll.
4. **Background threads and an in-memory store.** This keeps acknowledgment immediate with no extra infrastructure. The trade-off is that tasks don't survive a restart (see Limitations).
5. **The Requester tells the Specialist what the form accepts.** `allowed_categories` travels with the task, so the Specialist's answer is shaped for its purpose.
6. **Deterministic category mapping, done in code.** The LLM chooses the knowledge base's own category; code maps it onto the form's options by exact match. An Email or Security issue is never misfiled as, say, Software. It comes back as `category: null` with `kb_category` set, and the Requester routes it for manual handling. We chose accuracy over always producing a ticket.
7. **Section-level chunking.** The documents are short and consistently structured, so each section is a natural, self-contained unit. Fixed-size chunks would have split steps from their headings.
8. **Two relevance checks.** The similarity gate rejects off-topic questions before spending an LLM call. The LLM's answerable flag catches passages that pass the gate but don't answer the question. The gate uses cosine similarity, not rerank scores. Measured rerank scores didn't separate the cases: a real issue ("monitor flickering") scored -9.5 , only slightly above an off-topic one ("bake a cake") at -11.1 . Cosine similarity separated them cleanly: 0.28 vs 0.01 .
9. **Strict structured output with a per-request schema.** Groq enforces the schema while generating, rather than relying on the model to follow instructions. That's why the model was chosen: `gpt-oss-20b` supports strict mode, while the Llama models

in Groq's lineup support only best-effort JSON. The generator falls back to plain JSON mode plus validation if another model is configured.

10. **Local embeddings and reranker.** They're free, need no API key, and remove a network dependency from retrieval. Only generation uses an external API.
11. **Prompt tightened for grounding.** Early outputs added steps that aren't in the knowledge base, such as "forward the email to the security mailbox". The prompt now requires every action to come from the passages and asks for third-person technician notes.
12. **Accessible Playwright locators and explicit verification.** Locating fields by label and role is sturdier than relying on element IDs. Checking the echoed values, not just "a confirmation appeared", proves the Specialist's output actually reached the ticket.
13. **Mock app served over HTTP, left unchanged.** Serving it on a local port behaves like a real web application rather than a local file, and leaving it untouched respects it as the provided target system.
14. **Rule-based Requester.** The spec allows it, and it makes the coordination logic predictable and easy to explain. The interactive session lets one run handle many requests.
15. **Built-in fault injection.** /slow and /fail make the timeout and failure paths reproducible on demand for testing and demonstration.
16. **Console-safe output.** The LLM's typographic punctuation crashed Windows consoles during testing. Answers are normalized to plain ASCII, and output never crashes on unprintable characters.

Reflection

I built the A2A layer before the RAG pipeline, with a retrieval-only placeholder behind the Specialist's single `handle_task()` function. When the full RAG pipeline was ready, it replaced that one function and the protocol, the Requester and the Playwright code didn't change. It also meant I could test the whole workflow end to end, including every failure path, before the LLM was involved.

Both advanced RAG techniques gave measurable gains, not just theoretical ones. contextual chunk headers raised top-1 retrieval accuracy on the course test cases from 4/5 to 5/5. Reranking fixed a case vector search got wrong: "someone hacked my account"

retrieved Account Access instead of Security. Because I measured each technique on its own, I can say what each contributed rather than just that I used them.

Strict structured output removed a whole class of errors. Building the JSON schema for each request, so the category and citations can only come from the retrieved passages, meant the LLM never returned a category or source that didn't exist. The Requester always received data it could act on, with no text parsing.

Fault injection made the failure paths easy to demonstrate. The /slow and /fail switches let me show a timeout or a failed task on demand, instead of waiting for a real outage.

Challenges

The knowledge base defines six ticket categories, but the form offers only four; Email and Security are missing, and one of the course's own test cases expects Email. I had to decide whether to force those into the nearest form category or refuse to file them. I chose to refuse and route them for manual handling. A misfiled ticket is worse than an unfilled one, especially a security incident filed as "Account Access". The cost is that some valid requests never produce a ticket automatically.

Early answers were well written but sometimes went beyond the passages. The phishing answer told the technician to "forward the email to the security mailbox and delete it", and nothing like that appears in security.md. Tightening the prompt removed the invented steps, but a milder addition remained: "provide the user with escalation contact information". Prompt instructions reduce invented content; they don't guarantee grounding.

Rerank scores couldn't be used as the relevance cutoff. I first planned to use the reranker's score to reject off topic questions. Measurement showed its raw scores don't separate the cases. The cutoff uses cosine similarity instead, which separates them cleanly (0.28 vs 0.01).

RAG effectiveness

- Retrieval: 5/5 course test cases retrieve the correct category first.
- Full pipeline: 11/11 on the 5 test cases plus 6 edge cases. That's correct categories for all nine issues, including both Security cases and a vaguely worded "monitor flickering", and the two off-topic questions were rejected.
- Escalation: flagged only where the knowledge base's criteria apply: suspected compromise, phishing, and an office-wide outage.
- Speed: about 1 second per answer.

Agent communication

The submit, acknowledge and poll pattern worked reliably and is easy to observe. Every status change is time stamped. It has clear weaknesses:

- Polling is wasteful and adds delay. The Requester checks every second, so a result can wait up to a second before being collected. Most tasks finished in under a second, often between two polls, which is why I added the status history to show the working state at all.
- There's no way to cancel a task. When the Requester times out, the Specialist keeps working and completes a result nobody collects.
- Task state lives in memory. Restarting the Specialist loses every task.
- The client is fragile to brief network problems. A single failed status check aborts the task instead of retrying. A reply from the Specialist that doesn't match the expected format isn't handled cleanly.

Limitations

- Requests in the Email and Security categories can't be filed through the form.
- Tasks don't persist and can't be cancelled,
- The Requester's list of form categories is constant. Playwright rechecks it against the live page, but the Requester doesn't discover it automatically.
- Answers depend on an external API with rate limits.
- The Requester can't ask the user follow-up questions when a request is ambiguous.

Potential Improvements

- Push status updates instead of polling. Use server sent events or a webhook
- Add a POST cancel endpoint so abandoned tasks can be stopped.
- Persist tasks using a table or SQL database for memory
- Make the client resilient by retrying failed polls with backoff